BarberoBot

A Hybrid RAG Conversational Assistant for Semantic Digital Libraries

Martina Uccheddu
martina.uccheddu@studio.unibo.it

1 Introduction

BarberoBot is a conversational assistant utilizing a hybrid Retrieval-Augmented Generation (RAG) architecture. It was conceived as a complementary tool to Barberotheca, an existing semantic digital library that organizes the transcriptions of professor Alessandro Barbero's Festival della Mente lectures. The project addresses a specific gap: providing users with a transversal and intuitive way to explore the corpus.

Based on a user's query, Barberobot offers a general summary of relevant lessons. Furthermore, it suggests related lessons that share the same historical entities or macro-themes, thereby proposing new pathways for exploring the library. To bring the user experience full circle, the bot directly injects source links in its answers, seamlessly redirecting the user back to the Barberotheca platform.

2 Hybrid RAG Architecture

The system employs a hybrid RAG approach, integrating a semantic vector store with a structured RDF knowledge graph. Traditional vector embeddings are powerful for semantic matching but can occasionally miss direct entity hits. Graph-based RAG overcomes these limitations by utilizing relational structures to process domain-specific queries and multihop reasoning.

In Barberobot, the graph performs two critical roles:

- It analyzes the query to select precise lessons, acting as a grounding mechanism to ensure that specific fragments from those lessons are forced into the vector retrieval pool.

- It connects the generated response to strictly verified metadata, such as YouTube source links, Barberotheca URLs, and the correct names of related lessons.

This hybrid approach leverages Barberotheca's pre-existing Turtle (.ttl) knowledge graph. By grounding the retrieval in these structured relationships, the system mitigates the risk of large language models fabricating facts, drastically reducing hallucinations. If the bot is to effectively assist users in navigating a semantic digital library, its answers must be as truthful and directly tied to the source material as possible.

3 Implementation, Scalability and Chunking Strategy

The architecture is designed with modularity and long-term scalability in mind.

- **Ingestion**: The raw data is dynamically pulled from the Barberotheca Git submodule using a sparse-checkout method, isolating only the necessary transcripts and metadata.

- **Graph Store**: To avoid the heavy computational cost of parsing a large RDF graph on every startup, the system calculates an MD5 hash of the .ttl file. If the hash remains unchanged, it loads a highly optimized, cached .pkl version of the graph.

- **Vector Store**: The ChromaDB vector store utilizes a `DocstoreStrategy.UPSERTS` approach. This means the system keeps track of already processed documents; if a document is unmodified, it is skipped.

This architecture guarantees that the project is highly scalable. If Barberotheca adds new lessons or transcripts, BarberoBot can easily include them in its RAG corpus. By simply pulling the new submodule data and running the `vector_store.py` script, the system uses an 'upsert' strategy to cost-effectively embed only the newly added transcripts, while the Knowledge Graph cache automatically detects changes and regenerates itself upon the next application startup.

Before entering the vector store, the transcripts undergo a specific chunking process (`chunk_size=256, chunk_overlap=64`). This configuration is explicitly tailored to the nature of the source material: transcribed spoken lectures. Spoken language inherently features digressions, incidental phrases, and complex conversational flow. Determining optimal chunk length is a known challenge in RAG architectures; excessively small chunks risk fragmenting sentences, while overly large chunks can introduce irrelevant context. To accommodate Barbero's conversational delivery style, where he frequently inserts incidental phrases or refers back to earlier points, the system implements a sliding window approach with a substantial 64-token overlap. This ensures that the semantic thread is preserved across chunk boundaries, preventing crucial context from being severed during the retrieval phase.

4 The Hybrid Retrieval Pipeline

Once a user submits a query, the `SmartHybridRetriever` executes a carefully orchestrated pipeline (see figure 1).

The diagram illustrates the step-by-step execution of the `SmartHybridRetriever` pipeline. Initially, the system cleans the user's query of conversational fluff to extract its core semantic meaning. The knowledge graph then performs Named Entity Recognition (NER) on this cleaned query, identifying specific historical entities and flagging their corresponding lessons for "forced" retrieval. To optimize latency, the query's vector embedding is computed only once; it is then used to perform both a wide semantic search and a targeted search specifically on the graph-flagged lessons. After deduplicating overlapping text chunks, a Cohere cross-encoder reranker evaluates and scores all remaining candidates. Finally, chunks originating from the graph-boosted lessons receive an artificial score increase, guaranteeing their inclusion alongside injected graph metadata in the final context passed to the LLM.

By forcing the retrieval of lessons identified by the graph, the system guarantees that highly relevant entities are not lost, even if their purely semantic vector similarity was initially deemed low.

5 Model Selection: Reranker and LLM

The retrieval pipeline incorporates a dedicated reranking phase. Passing retrieved documents through a cross-encoder model (in this case, Cohere) is a proven best practice in RAG workflows to significantly augment the relevance of the retrieved chunks before they reach the generator.

For the generation phase, the system utilizes the `qwen/qwen3.8-27b` large language model configured via the Groq API. Despite the architectural trade-off of requiring an extra external API key insertion, this specific model setup was retained because it strictly adheres to a low-temperature, deterministic constraint (`temperature=0.1`) and boasts a massive 32,768 token context window capable of processing all retrieved transcripts and graph metadata.

In knowledge-intensive domains like the Humanities and the broader GLAM (Galleries, Libraries, Archives, and Museums) sector, factual accuracy and reliable source use are absolute priorities.

The Barberobot architecture mitigates the LLM hallucinations issue by forcing the Qwen model to ground its answers strictly in the retrieved historical data, rather than relying on its own pretrained, latent knowledge. The combination of a highly deterministic LLM configuration and a robust Retrieval-Augmented Generation (RAG) framework ensures faithful context reproduction, providing users with a conversational assistant that is as truthful and source-reliant as possible.

6 Custom Graph RDF vs. Standard Frameworks

During development, the decision was made to build a custom RDF retrieval function rather than relying on LlamaIndex's built-in `KnowledgeGraphIndex`.

Standard framework tools are often designed to construct graphs from scratch out of text documents, but they struggle to easily ingest and generate embeddings for existing graphs. By creating a `CustomRDFRetriever`, the system bypasses heavy full-graph scans. It builds optimized, in-memory indices that allow for instant lookups of entities and their related lessons, vastly improving query latency.

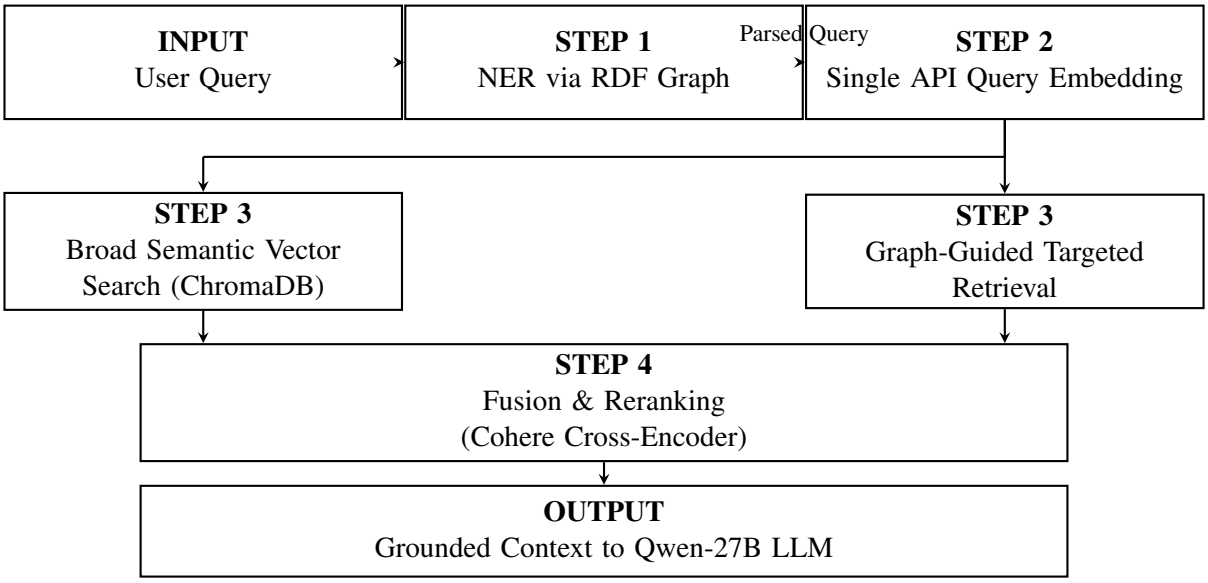

Figure 1: SmartHybridRetriever Pipeline Architecture

7 Application UI and Limitations

The user interface is built using Chainlit, which was highly customized to visually and functionally integrate with the main project. To reinforce Barberobot's role as a direct extension of the Barberotheca platform, the UI mirrors its parent project's aesthetic. The application defaults to a dark theme and incorporates the exact same fonts, color palette, and official logo as the main site. This ensures a seamless and visually cohesive transition for users moving between the semantic library and the conversational assistant.

Functionally, the interface is designed to transmit transparency and grounding through two primary chat features:

- **Transparent Sourcing**: Whenever Barberobot answers, it displays the exact transcription segments used to generate the response in a dedicated side panel. This ensures the user can independently verify the AI's claims.
- **Direct Archive Integration**: Responses automatically append a "Sources" section containing direct, clickable links to the Barberotheca website. This allows users to seamlessly open, read, or listen to the original lesson, bringing the exploration journey full circle.

However, the current UI does present a few minor limitations that slightly impact the overall user experience:

- **Forced Side Panel**: Because of how the transparent sourcing is implemented, Chainlit automatically forces the side panel open to display the transcript chunks. This behavior cannot be easily disabled and creates a slightly cluttered user experience upon generation.
- **Lack of Streaming**: The current implementation does not utilize token streaming. While the total response time is efficient, loading the output entirely at once rather than word-by-word slightly disrupts the UX illusion of real-time conversational generation.

8 Potential Further Improvements

While Barberobot currently uses a static corpus, a valuable future improvement would be supporting dynamic file uploads. Users could upload personal lesson notes or PDFs, transforming the tool into a personalized study assistant.

Technically, RAG architectures can rapidly adapt to new tasks without updating model parameters, as long as query-related documents are provided. Since the backend already utilizes LlamaIndex, temporarily vectorizing user notes to cross-reference against the Barbero corpus would be highly feasible. This expansion aligns perfectly with core RAG use cases, such as specialized question-answering and fact-checking in educational contexts.

References

IBM, *Cos'è GraphRAG?*. IBM Think, https://www.ibm.com/it-it/think/topics/graphrag

Kenneth Enevoldsen, et al., *MMTEB: Massive Multilingual Text Embedding Benchmark*. arXiv, https://arxiv.org/abs/2502.13595, 2025.

Jay Kim, *Hybrid Retrieval-Augmented Generation (RAG): A Practical Guide*. Medium, https://medium.com/@bravekjh/hybrid-retrieval-augmented-generation-rag-a-practical-guide-dab74fc28ee9, 2025.

Tom Krantz and Alexandra Jonker, *Database vettoriali RAG, definizione*. IBM Think, https://www.ibm.com/it-it/think/topics/rag-vector-database,

Xiaiyao Wang, et al., *Searching for Best Practices in Retrieval-Augmented Generation*. arXiv, https://arxiv.org/pdf/2407.01219, 2024.

Aston Zhang, et al., *Introduction*. Dive into Deep Learning, https://d2l.ai/chapter_introduction/index.html,